

CSD-380: DevOps

Module 3: Version Control

Assignment 2: Version Control Guidelines

Isaac Ellingson

6/21/2026

For my research, of the articles I found available, I chose ones by GitHub, GitLab, and Perforce. GitHub and GitLab are both providers and users of version control, and specifically of git which is also what I use. They also necessarily, as part of improving their product, speak to many developers to learn about their needs and struggles with the product, and with version control in general. They're both also in touch with the needs of small, indie developers. I will listen to the advice of such organizations. Perforce is more of an enterprise shop that handles a *lot* of DevOps concerns, but they make a proprietary "Perforce P4" version control system, so they may have good advice for large-scale systems.

GitHub

Article main points:

- Give each user only the minimum privilege they need, and incorporate backups and failovers into the system.
- Write clear commit messages.
- Make frequent, small commits, but only when each change is tested and working.
- Pull new code frequently, especially before starting new work yourself.
- Use branches to work on different features and releases.

Their advice stresses good security, communication, and conflict avoidance. From my own experience, "Commit early, commit often" is an extremely good policy.

For advice that's no longer relevant, I would say "use branches" fits the bill. It used to be good advice, but Agile is moving away from that as a strategy because it undercuts the principle that "every change should be releasable". You're adding time and overhead to getting that code to production because it can be difficult and time-consuming to integrate multiple branches back into the trunk, and the goal would be what, collecting several changes into one focused update?

Instead, for my own work, I drop a tag at each version release. This means it's easy to checkout the code for a particular version a user's having trouble with, build it, and put it into a test environment to reproduce issues, diagnose root causes, and find workarounds. Then, we can also discover if the problem has been solved by a newer version (in which case the long-term fix is to update the software), or whether new cards need to be added to fix these issues going forward.

GitLab

Article main points:

- For each problem, find a minimal fix and make *small*, focused changes in each commit. Don't put two unrelated things in a commit, because that inflates the commit.
- Use branches to keep unstable code separate from production code. Use a branching strategy so everyone uses and understands branches in the same way.
- Write clear, descriptive commit messages.
- Use code reviews to ensure code quality.

Once again, there's great advice here. It's important to have a repository maintainer who is responsible for project hygiene (I imagine this is frequently the Product Owner). In previous classes we've talked about logging for security purposes, and how the logging does you no good if you never review it - same thing here. Your git history isn't helping you if you don't use it. At a minimum, every pull request (a request to merge a fork back into main, or to contribute code to an open-source project) needs a code review. These code reviews work wonders not just to keep the code quality of the project high, but also when receiving them to improve yourself as a programmer. I've had code reviews that were *very painful* but resulted in immense personal growth for me. Now my goal is to hold the same standard, but be less cruel about it!

And once again, there's this idea of using branches for sandboxing, to keep unstable code out of main. I'm seeing some merit in GitLab's framing, as the pull request gives you an opportunity for code review on the feature as a whole, but ideally you want to bake that quality control into the dev process earlier! A large team might do pair programming, for example, to achieve many of the same benefits.

Perforce

Article main points:

- Make small commits that encompass all the changes for one small improvement.
- Only commit when you have made an improvement or solved a problem - don't commit partial work.
- Clearly document your commits, write good messages for them.
- Make every commit usable / releasable. Don't break something with a commit.
- Do code reviews to maintain and improve software quality.
- Include test cases with your changes, and make sure the author of the change is logged.
- Use branches for releases or milestones, and to protect your main from faulty or experimental code. Use a branching strategy to avoid conflicts and inconsistencies across the project.
- Incorporate security such as access control, backup and failover, and logging.

There's a lot of similar advice in here. I like the idea that the authors that implement a card also write at least one test for that card. It serves both to help validate the code and to discover and document its corner cases.

I also think it's interesting that Perforce goes much more in depth about what should be in a commit, what git should be used for essentially. I have often said, "don't use git like dropbox," and what I meant was, git is not a backup. You don't just throw your current work-state at it at the end of the day, or drop a new folder down at the end of each module. A Git repository is used to manage steady improvement of **one** project, and each commit should encompass all the changes you made, and address a single cognizable problem or missing feature in the system. If you think about it, how would you write a meaningful commit message for anything else?

Perforce talks about branching yet again, but they do at least acknowledge that branching can cause issues that take a lot of velocity out of the system. Unlike other articles, they stress keeping the branches very simple, giving each branch an *owner*, and merge-down-copy-up (which in practice is what everybody does, but it's good advice nonetheless, and I imagine some people might *not* see it as very obvious).

Similarities and Differences

There is broad agreement between these articles. We see time and again advice to think about security and resilience, make frequent, small, focused commits, write good messages for those commits, and to develop habits and policies to prevent conflicts. As expected, GitHub and GitLab are more focused advice for small projects. Perforce as seems to have both more advice in total, and more advice about security, workflow, and policy.

My Own Guidelines

Broadly, I agree with the sources:

Secure and Protect Your Version Control System

Designate a Maintainer, typically the product owner. Per the principle of least privilege, give each contributor only the access they need. Ensure commits are signed and authorship is recorded. GitHub offers a simple switch to place repositories into a relatively secure mode, denying unverified commits and preventing many dangerous actions like force-pushing.

Whether your repository is a service or self-hosted, ensure that it has backups and fail-over built-in. It is very expensive to close your entire software shop for a day, but it's really hard to get anything at all done without your version control.

Use Version Control for Its Strengths

Even in our current classes, even in the programming classes, we aren't using git and GitHub in a productive way. It is **only** wasting time, because it's functioning as a folder backup. We have better, more focused systems available for that, like iCloud, Microsoft OneDrive, DropBox, and Proton Drive, where you can create persistent storage that's automatically replicated across devices you work on, and even doubles as a remote backup. Git is for managing *changes* to a project over time, especially when more than one collaborator is working on it. So, don't use version control where it doesn't make sense. Do use it where it makes sense.

When you do use version control, play to its strengths. At any step in the process, your collaborators will want to be able to pull changes down to their local workspace, and then get to work immediately on new changes they want to contribute. For that reason, don't commit nonworking code. Each commit should include every part of a complete improvement, in working order and tested. It should be clearly labeled, so that if it causes problems (at least in git,) you can easily find it and use a cherry-pick to remove it, even if changes were committed after it. Each commit should also avoid changing unrelated things - for example, if formatting is changing, it had better be a commit with only formatting changes in it!

To make all this easier, you plan small changes that move you incrementally closer to where you want to go. Part of implementing a large feature should already be splitting it up into pieces that you can plan and accomplish. Just keep splitting till you have a nice, lean commit.

Use Code Review in Your Process

Version control, especially as embodied in software forges, give you wonderful tools to incorporate code reviews into your workflow. Use those tools! In GitHub, for example, you can attach comments to individual lines or groups of lines and request changes or clarification. You can tag in more reviewers if needed. The repository maintainer should be really engaged in this process, offering advice on how to best achieve the overarching project goals and enforcing uniform standards across the codebase.

Commit Early and Often

Going hand in hand with making small, focused, atomic commits is just that the sooner code is out there in the repository, the sooner you can incorporate advice from your colleagues, and the more lead time you might have on adapting to changes someone else is working on. Communication is a big part of effective software development. Your commits are part of that communication. Don't clam up. Don't hide your ideas just because they seem awkward now. Keep talking and it's possible someone else, with a different point of view, will happen to have the key to making everything work cleanly.

If You're Going to Use Branches, Keep Them Simple

When you make a branch, think ahead: This branch is eventually going to be a Pull Request. A PR is subject to a code review, and as such it should be a focused, minimal endeavor to

make one improvement to the codebase. This can operate at a larger scale than a commit, but devs should be extremely vigilant about scope creep in branches, and maintainers should hold them accountable for it. A simple branch with a focused objective can also help a team achieve greater autonomy than one whose scope is unclear.

If you must use branches, having branching policies can help teams work across branches, understand what is (and should be!) in a branch, and have reasonable expectations about how long a branch should live before getting merged back in. Expectations are important for a branch so that a silo doesn't wander off into creeping elegance and never reel itself in. Remember, DevOps shouldn't silo that much!

That's my list. I didn't change a whole lot at all from the source material, and attempted to incorporate **all** the good advice I found. I don't speak as much about branches and I don't speak as glowingly as the other sources because I think we're moving away from that strategy as an industry for good reasons. I left out a lot of Perforce advice because some things are just enterprise concerns at the end of the day, like logging, protecting your IP, etc. - also, using words like "atomicity", or "ACID" make sense to me, but make for a much denser, less skimmable, and probably less useful list. I can speak more authoritatively about small to mid-sized projects and wanted to speak to that developer in my list.

References

GitHub. (2025, October 15). What is version control? Retrieved June 22, 2026, from GitHub

website: <https://github.com/resources/articles/what-is-version-control#best-practices-for-version-control>

Gitlab. (n.d.). What are Git version control best practices? Retrieved from GitLab website:

<https://about.gitlab.com/topics/version-control/version-control-best-practices/>

Schiestl, B. (2020, May 21). 8 version control best practices. Retrieved from Perforce

Software website: <https://www.perforce.com/blog/vcs/8-version-control-best-practices>